

姓名	李泽浩	学号	2025218895	实验成绩	
专业班级	软件工程 25-9 班	实验时间	4.19	实验地点	新安学堂

实验一 顺序表实验

1 实验目的

- 1) 熟练掌握线性表的顺序存储结构。
- 2) 熟练掌握顺序表的有关算法设计和实现。
- 3) 根据具体问题的需要，设计出合理的表示数据元素的顺序结构，并设计相关算法。

2 实验要求

- 1) 顺序表结构和运算定义，算法的实现以库文件方式实现，不得在测试主程序中直接实现；比如存储、算法实现放入文件：seqList.h
- 2) 程序有适当的注释，程序的书写要采用缩进格式。
- 3) 实验程序有较好可读性，各运算和变量的命名直观易懂，符合软件工程要求；
- 4) 程序要具有一定的健壮性，即当输入数据非法时，程序也能适当地做出反应，如插入删除时指定的位置不对等等。
- 5) 程序要做到界面友好，在程序运行时用户可以根据相应的提示信息进行操作。
- 6) 程序运行、测试正确；
- 7) 根据实验报告模板详细书写实验报告。

3 实验任务

编写算法实现下列问题的求解。

- 1) 在一个递增有序的顺序表 L 中插入一个值为 x 的元素，并保持其递增有序特性。实验测试数据基本要求：

顺序表元素为 (10,20,30,40,50,60,70,80,90,100),
x 分别为 25, 85, 110 和 8

- 2) 将顺序表 L 中的奇数项和偶数项结点分解开（元素值为奇数、偶数），分别放入新的顺序表中，然后原表和新表元素同时输出到屏幕上，以便对照求解结果。实验测试数据基本要求：

第一组数据：顺序表元素为 (1,2,3,4,5,6,7,8,9,10,20,30,40,50,60)

第二组数据：顺序表元素为 (10,20,30,40,50,60,70,80,90,100)

- 3) 求两个递增有序顺序表 L1 和 L2 中的公共元素，放入新的顺序表 L3 中。

实验测试数据基本要求：

第一组

第一个顺序表元素为 (1, 3, 6, 10, 15, 16, 17, 18, 19, 20)

第二个顺序表元素为 (1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 18, 20, 30)

第二组

第一个顺序表元素为 (1, 3, 6, 10, 15, 16, 17, 18, 19, 20)

第二个顺序表元素为 (2, 4, 5, 7, 8, 9, 12, 22)

第三组

第一个顺序表元素为 ()

第二个顺序表元素为 (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

- 4) 删除递增有序顺序表中的重复元素，并统计移动元素次数，要求时间性能最好。实验测试数据基本要求：

第一组数据：顺序表元素为 (1,2,3,4,5,6,7,8,9)

第二组数据：顺序表元素为 (1,1,2,2,2,3,4,5,5,5,6,6,7,7,8,8,9)

第三组数据：顺序表元素为 (1,2,3,4,5,5,6,7,8,8,9,9,9,9)

4 顺序表扩展实验*

(非必做内容，有兴趣的同学选做)

- 1) (递增有序) 顺序表表示集合 A、B，实现：

$C=A \cup B$, $C=A \cap B$, $C=A-B$

$A=A \cup B$, $A=A \cap B$, $A=A-B$

- 2) 一个长度为 L ($L \geq 1$) 的升序序列 S，处在第 $\lceil L/2 \rceil$ 个位置的数称为 S 的中位数。例如，若序列 S1=(11, 13, 15, 17, 19)，则 S1 的中位数是 15。两个序列的中位数是含它们所有元素的升序序列的中位数。例如，若 S2=(2, 4, 6, 8, 20)，则 S1 和 S2 的中位数是 11。

现有两个等长升序序列 A 和 B，试设计一个在时间和空间两方面都尽可能高效的算法，找出两个序列 A 和 B 的中位数。要求：

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想，采用 C 或 C++ 语言描述算法，关键之处给出注释。
- (3) 说明你所设计算法的时间复杂度和空间复杂度。

5 算法设计与实现描述

(书上给出的基本运算外，其它问题必须先给出算法思想或步骤，再给出算法描述)

```
#ifndef SEQLIST_H
#define SEQLIST_H

#include <iostream>

struct list {
    int data[100];
    int size;
    // 构造函数：初始化 size
    list() {
        size = 0;
    }
    // 析构函数
    ~list() {
    }
    // 成员函数：基础插入（直接放到尾部）
    void insert(int value) {
        if (size < 100) {
            data[size] = value;
            size++;
        } else {
            std::cout << "表已满" << std::endl;
        }
    }
};

#endif
```

```

    }
}

// 成员函数: 打印输出
void display() {
    for (int i = 0; i < size; i++) {
        std::cout << data[i] << " ";
    }

    std::cout << std::endl;
}
};
#endif

```

上述代码定义了一个顺序表 `seqlist`。包括了构造，析构，打印，插入四个功能。

(1) 在一个递增有序的顺序表中插入一个元素，并保持其递增有序特性

【算法思想】

首先判断顺序表是否已满。若不满，则利用循环从前向后遍历顺序表，寻找第一个大于待插入元素的位置。找到后，将该位置及其之后的所有元素整体向后移动一位，为新元素腾出空间。最后将新元素赋值给该位置，并将顺序表的长度属性加一。

【算法步骤】

- ① 溢出检查：判断当前顺序表长度是否已达到最大容量。若达到，则输出提示并结束操作。
- ② 极值判断：若待插入数值小于等于表头元素，则全体后移并插入首位；若大于等于表尾元素，则直接追加到末尾。
- ③ 遍历寻址：使用循环从头向后比较，找到第一个比待插入数值大的元素所在的位置。
- ④ 移位与插入：将寻找到的位置及以后的元素全部向后移动一位，将待插入元素填入该位置，并更新顺序表长度。

【算法描述和实现】

```

#include <iostream>
#include "seqlist.h"
using namespace std;
void insertlist(list &L, int value){
    if(L.size >= 100){
        cout << "顺序表已满 " << value << "." << endl;
    }
    else if (value <= L.data[0])
    {
        for(int i=L.size;i>0;i--){
            L.data[i]=L.data[i-1];
        }
        L.data[0]=value;
        L.size++;
    }
    else if (value >= L.data[L.size-1])
    {
        L.data[L.size] = value;
        L.size++;
    }
}

```

```

    }
    else {
        for (int i = 0; i <= L.size; i++)
        {
            if (value < L.data[i])
            {
                for (int j = L.size; j > i; j--)
                {
                    L.data[j] = L.data[j-1];
                }
                L.data[i] = value;
                L.size++;
                break;
            }
        }
    }
}
}
}

```

(2) 奇偶数分开

【算法思想】 准备两个额外的顺序表，分别用来存放分离出来的奇数和偶数。接着遍历原始顺序表，逐个取出其中的数据。对每一个取出的数据，通过判断它能否被二整除来确认奇偶性。如果能被整除，就认定它是偶数并插入到偶数表中；反之则是奇数，插入到奇数表中。最后依次调用显示函数展示结果。

【算法步骤】

- ① 初始化：创建两个新的顺序表 `oddList` 和 `evenList`，分别用于存储奇数和偶数。
- ② 遍历判断：利用循环遍历原始顺序表，对每个元素使用取模运算判断其奇偶性。
- ③ 分发插入：若为偶数则调用插入函数加入 `evenList`；若为奇数则加入 `oddList`。
- ④ 结果输出：分别调用展示函数打印原始表、奇数表和偶数表的内容。

【算法描述和实现】

```

#include <iostream>
#include <vector>
#include "seqlist.h"
using namespace std;
void Distinguish(list L){
    list oddList; // 存储奇数的顺序表
    list evenList; // 存储偶数的顺序表
    for (int i = 0; i < L.size; i++) {
        if (L.data[i] % 2 == 0) {
            evenList.insert(L.data[i]); // 插入偶数到偶数表
        } else {
            oddList.insert(L.data[i]); // 插入奇数到奇数表
        }
    }
}
cout << "原始表: ";

```

```

L.display(); // 输出原始表

cout << "奇数表: ";

oddList.display(); // 输出奇数表

cout << "偶数表: ";

evenList.display(); // 输出偶数表

}

```

(3)求两个递增有序顺序表中的公共元素，放入新的顺序表中

【算法思想】 为了达到更高的运行效率，这里采用双指针策略。设置两个指针分别指向两个顺序表的开头。只要这两个指针都没有走到各自表的末尾，就不断对比它们当前指向的数据：如果两个数据相同，说明找到了公共元素，存入新表，同时让两个指针都往后走一步；如果某一个指针指向的数据较小，说明该数据不可能是公共元素，就只把该指针往后移去寻找更大的数。这种方法避免了繁琐的嵌套比对。

【算法步骤】

- ①初始化：创建一个新的顺序表 `publicList` 用于存储交集，并初始化两个指针 `i` 和 `j` 分别指向起始位置。
- ②循环遍历：在两个指针均未越界的前提下，开启循环对比当前指向的元素大小。
- ③移动与提取：若两元素相等，将其存入 `publicList` 且两指针同时后移；若前者的元素较小，则只让前者的指针后移；反之则让后者的指针后移。
- ④结果检验：遍历结束后，检查 `publicList` 的长度并分类输出最终结果。

【算法描述和实现】

```

#include <iostream>
#include <vector>
#include "seqlist.h"
using namespace std;

void insertlist(list &L, vector<int> value){
    for (int i = 0; i < value.size(); i++) {
        L.insert(value[i]);
    }
}

void Publicelement(const list &L1, const list &L2){
    list publicList;
    int i = 0, j = 0;
    while (i < L1.size && j < L2.size) {
        if (L1.data[i] == L2.data[j]) {
            publicList.insert(L1.data[i]);
            i++; // 相等时，两个指针一起后移
            j++;
        } else if (L1.data[i] < L2.data[j]) {
            i++; // L1 的元素小，L1 的指针后移去寻找更大的
        } else {
            j++; // L2 的元素小，L2 的指针后移
        }
    }
}

```

```

    cout << "公共元素表: ";
    if (publicList.size == 0) {
        cout << "无" << endl; // 如果没找到, 明确告诉用户“无”
    } else {
        publicList.display(); // 找到了就正常打印
    }
}

```

(4) 删除递增有序顺序表中的重复元素, 要求时间性能最好

【算法思想】 采用快慢双指针的方法直接在原表内部进行去重操作, 从而避开大量数据频繁往前挪动的耗时过程。慢指针一开始停留在第一个数据的位置, 快指针从第二个数据开始往后挨个查看。只要快指针发现自己指向的数据和慢指针指向的数据不一样, 就让慢指针往前走一步腾出地方, 然后把快指针找到的新的数据复制过去覆盖掉原来的值。最后只需根据慢指针走过的总步数重新确定顺序表的长度即可。

【算法步骤】 ① 边界处理: 检查顺序表长度, 若长度不超过一则必定不存在重复情况, 直接输出并返回。

② 初始化双指针: 慢指针 `slow` 停留在首个元素位置, 快指针 `fast` 从第二个元素位置开始向后探路。

③ 探路与覆盖: 快指针向后循环遍历, 若遇到与慢指针不同的新的数值, 则慢指针前进一步腾出新位置, 并将快指针的数据覆盖过去。

④ 更新表长: 遍历结束后, 将顺序表的长度更新为慢指针当前步数加一, 并输出去重后的结果。

```

#include <iostream>
#include <vector>
#include "seqlist.h"
using namespace std;

void insertlist(list &L, vector<int> value){
    L.size = 0; // 确保在插入前清空顺序表
    for (int i = 0; i < value.size(); i++) {
        L.insert(value[i]);
    }
}

void deleteelement(list &L) {
    if (L.size <= 1) {
        cout << "删除重复元素后的表: ";
        L.display();
        return; // 只有 0 或 1 个元素, 必定不重复, 直接返回
    }

    int slow = 0; // 慢指针停在第一个元素
    for (int fast = 1; fast < L.size; fast++) { // 快指针从第二个元素开始探路
        // 如果快指针找到了和慢指针不一样的新鲜数字
        if (L.data[fast] != L.data[slow]) {
            slow++; // 慢指针前进一步, 腾出新位置
            L.data[slow] = L.data[fast]; // 把新数字赋值过去
        }
    }
}

```

```

    }

    L.size = slow + 1;

    cout << "删除重复元素后的表: ";

    L.display();
}

```

顺序表拓展实验

(1)

【算法思想】

利用两个顺序表均已按照从小到大排序的特性，采用双指针法来高效求解集合运算。对于交集，设置两个游标分别从表 A 和表 B 的头部开始遍历。若两个游标指向的数据相同，则将该数据存入新表，同时两个游标一起向后移动；若数据不同，则让数据较小的那个游标单独向后移动去寻找更大的值。

对于并集，同样使用双游标比对。若数据相同，只提取其中一个存入新表，两游标同时后移；若数据不同，则将较小的那个数据存入新表，并让对应的游标后移。当其中一个表被遍历完后，将另一个表剩余的所有数据依次追加到新表尾部。

对于只在 A 中存在，不在 B 中存在的数，在游标比对时，若数据相同，说明该数据同时存在于两表中，予以舍弃，两游标同时后移；若 A 的数据较小，说明该数据是 A 独有的，将其存入新表并让 A 的游标后移；若 B 的数据较小，则单纯让 B 的游标后移。需要将运算结果直接作用于原表 A 时，只需先利用上述逻辑生成新表 C，最后将新表 C 的内容完整覆盖回表 A 即可。

【算法步骤】

- ①初始化：在各类集合运算函数中，均创建游标 i 指向表 A 的头部，游标 j 指向表 B 的头部，并创建新表 C 用于暂存结果。
- ② 并集处理：在循环中持续提取 A 和 B 当前游标下的较小值插入表 C，并移动对应游标。若遇到相同值则只提取一次。循环结束后，清理任意一个表中可能残留的剩余数据。
- ③ 交集处理：仅当游标 i 和 j 所指数据完全一致时，才将数据插入表 C 并双双后移；遇到大小不一的情况，仅移动较小数据所在的游标。
- ④ 差集处理：当游标 i 的数据小于游标 j 的数据时，将游标 i 的数据插入表 C。遇到相同数据则跳过不插入。最后将表 A 中可能剩余的数据全部追加到表 C 中。
- ⑤ 数据回写：在专门的覆盖函数中，调用上述逻辑得出结果表 C，再通过循环将表 C 的数据和长度属性逐一赋值给原表 A。

【算法描述和实现】

```

#include <iostream>
#include "seqlist.h"
using namespace std;
// 核心逻辑 1: 求并集 C = A ∪ B
list unionList(const list &A, const list &B) {
    list C;
    int i = 0, j = 0;
    while (i < A.size && j < B.size) {
        if (A.data[i] == B.data[j]) {
            C.insert(A.data[i]);
            i++;
            j++;
        }
    }
}

```

```

        } else if (A.data[i] < B.data[j]) {
            C.insert(A.data[i]);
            i++;
        } else {
            C.insert(B.data[j]);
            j++;
        }
    }

    // 将剩余元素追加到末尾
    while (i < A.size) { C.insert(A.data[i]); i++; }
    while (j < B.size) { C.insert(B.data[j]); j++; }

    return C;
}

```

// 核心逻辑 2: 求交集 $C = A \cap B$

```

list intersectionList(const list &A, const list &B) {
    list C;
    int i = 0, j = 0;
    while (i < A.size && j < B.size) {
        if (A.data[i] == B.data[j]) {
            C.insert(A.data[i]);
            i++;
            j++;
        } else if (A.data[i] < B.data[j]) {
            i++;
        } else {
            j++;
        }
    }

    return C;
}

```

// 核心逻辑 3: 求差集 $C = A - B$

```

list differenceList(const list &A, const list &B) {
    list C;
    int i = 0, j = 0;
    while (i < A.size && j < B.size) {
        if (A.data[i] == B.data[j]) {
            i++;
            j++;
        } else if (A.data[i] < B.data[j]) {
            C.insert(A.data[i]);
            i++;
        } else {
            j++;
        }
    }
}

```



```

    }
}

// 将 A 中剩余元素追加到末尾
while (i < A.size) { C.insert(A.data[i]); i++; }

return C;
}

// 附加逻辑：实现 A = A op B 的原地覆盖
void assignUnion(list &A, const list &B) {
    list C = unionList(A, B);
    A = C; // C++ 中同类型结构体可以直接赋值，覆盖原表
}

void assignIntersection(list &A, const list &B) {
    list C = intersectionList(A, B);
    A = C;
}

void assignDifference(list &A, const list &B) {
    list C = differenceList(A, B);
    A = C;
}

```

(2)

【算法思想】 已知两个序列长度均为大写字母 L ，合并后的总长度即为两倍的 L 。根据中位数的定义，合并后升序序列的中位数恰好位于第 L 个位置上。为了追求极高的空间效率，我们不需要真正申请一个新的数组去合并它们，而是采用“虚拟合并”的双游标策略。设置两个游标分别在表 A 和表 B 的开头待命。利用一个循环精准地执行 L 次跨步操作。在每一次跨步中，对比两个游标当前指向的数据，将较小的那一个数据记录到临时变量中，并让该游标向后推进一步。当循环严格执行完 L 次之后，临时变量中最后记录的那个数值，就是合并后序列的第 L 个元素，即所求的中位数。

【算法步骤】

- ① 防御检查：首先核对表 A 和表 B 的长度属性是否一致。如果不一致，则说明输入不符合等长序列的前提条件，向系统返回异常标识。
- ② 状态初始化：定义游标 i 和游标 j 均从零号位置起步。定义变量 `current` 用于实时存放当前步数下找到的较小数值。提取表 A 的总长度作为目标步数 L 。
- ③ 虚拟合并推进：启动一个专门执行 L 次的循环。在每一次循环中，安全地比较表 A 和表 B 在游标位置的数据大小。
- ④ 数据提取：若表 A 的数据更小，则将该数据更新给 `current` 变量，同时游标 i 后移；反之则将表 B 的数据更新给 `current` 变量，游标 j 后移。
- ⑤ 结果返回：循环自然结束后，直接输出 `current` 变量内部保留的最终数值。

【算法描述和实现】

```

#include <iostream>
#include "seqlist.h"
using namespace std;

int findMedian(const list &A, const list &B) {
    //时间复杂度 O(n)，空间复杂度 O(1)
    // 确保两个顺序表等长

```

```

if (A.size != B.size || A.size == 0) {
    cout << "输入序列不合法或不等长";
    return -1;
}

int i = 0, j = 0; // A 和 B 的当前索引
int current = 0;
int L = A.size;

// 合并后的中位数恰好是第 L 个被提取的较小元素
// 因此只需进行 L 次步进比对
for (int step = 0; step < L; step++) {
    // 如果 A 的元素较小, 或者 B 已经遍历完
    if (i < A.size && (j >= B.size || A.data[i] < B.data[j])) {
        current = A.data[i];
        i++;
    } else {
        // 否则取 B 的元素
        current = B.data[j];
        j++;
    }
}

return current;
}

```

6 运行结果截图及说明

(1)

```

//测试代码
int main(){
    list L;
    int values[] = {10,20,30,40,50,60,70,80,90,100};
    for (int i = 0; i < 10; i++)
    {
        L.insert(values[i]);
    }
    int x[]={25, 85, 110, 8};
    for (int i = 0; i < 4; i++)
    {
        insertlist(L,x[i]);
    }
    L.display();
}

PS D:\expt> & "c:\Users\34979\.vscode\extensions\ms-vscode.cpptools-1.31.4-win32-x64\debugAdapters\bin\WindowsDebugLaun
cher.exe" "--stdin=Microsoft-MIEngine-In-enxgoblz.zww" "--stdout=Microsoft-MIEngine-Out-qr5vmxzc.h0r" "--stderr=Microsof
t-MIEngine-Error-lbxorf2k.wzn" "--pid=Microsoft-MIEngine-Pid-naavrsas.ico" "--dbgExe=D:\vscode\mingw64\bin\gdb.exe" "--i
nterpreter=mi"
8 10 20 25 30 40 50 60 70 80 85 90 100 110

```

说明：程序成功将 25、85、110 和 8 插入到原顺序表（10,20...100）中。最终输出的表依然保持严格的递增顺序，验证了寻找插入位置及后移逻辑的正确性。

(2)

```
int main(){
    list L1, L2;
    vector<int> values1= {1,2,3,4,5,6,7,8,9,10,20,30,40,50,60};
    vector<int> values2= {10,20,30,40,50,60,70,80,90,100};
    for (int i = 0; i < values1.size(); i++) {
        L1.insert(values1[i]);
    }
    for (int i = 0; i < values2.size(); i++) {
        L2.insert(values2[i]);
    }
    cout<<"第一组数据:"<<endl;
    Distinguish(L1);
    cout<<"第二组数据:"<<endl;
    Distinguish(L2);
    return 0;
}
```

原始表: 1 2 3 4 5 6 7 8 9 10 20 30 40 50 60
 奇数表: 1 3 5 7 9
 偶数表: 2 4 6 8 10 20 30 40 50 60
 第二组数据:
 原始表: 10 20 30 40 50 60 70 80 90 100
 奇数表:
 偶数表: 10 20 30 40 50 60 70 80 90 100

说明：对包含 15 个元素的连续递增表和 10 个元素的递增表分别进行了测试。程序成功将原表中的奇数和偶数完美分离并分别存入新表，输出结果与预期一致。

(3)

```
int main(){
    list L1, L2;
    vector<int> values1= {1,3,6,10,15,16,17,18,19,20};
    vector<int> values2= {1,2,3,4,5,6,7,8,9,10,18,20,30};
    insertlist(L1, values1); // 插入第一组数据到 L1
    insertlist(L2, values2); // 插入第二组数据到 L2
    cout <<"第一次测试:"<<endl;
    Publicelement(L1, L2);
    L1.size = 0; // 清空 L1
    L2.size = 0; // 清空 L2
    cout <<"第二次测试:"<<endl;
    values1={1,3,6,10,15,16,17,18,19,20};
    values2={2,4,5,7,8,9,12,22};
    insertlist(L1, values1);
    insertlist(L2, values2);
    Publicelement(L1, L2);
    L1.size = 0;
    L2.size = 0;
    cout <<"第三次测试:"<<endl;
    values1={};
    values2={1,2,3,4,5,6,7,8,9,10};
    insertlist(L1, values1);
    insertlist(L2, values2);
    Publicelement(L1, L2);
}
```

第一次测试:
 公共元素表: 1 3 6 10 18 20
 第二次测试:
 公共元素表: 无
 第三次测试:
 公共元素表: 无

说明：程序准确地找出了第一组数据的交集（1 3 6 10 18 20）。而在处理完全不相交的第二组数据和包含空表的第三组数据时，均按预期输出“无”，边界异常情况处理得当。

(4)

```
int main(){
    list L;
    vector<int> values1= {1,2,3,4,5,6,7,8,9};
    cout<<"第一次测试:"<<endl;
    insertlist(L, values1);
    deletelement(L);
    cout<<"第二次测试:"<<endl;
    vector<int> values2= {1,1,2,2,2,3,4,5,5,5,6,6,6,7,7,8,8,9};
    insertlist(L, values2);
    deletelement(L);
    cout<<"第三次测试:"<<endl;
    vector<int> values3= {1,2,3,4,5,5,6,7,8,8,9,9,9,9,9};
    insertlist(L, values3);
    deletelement(L);
}
```

第一次测试:
 删除重复元素后的表: 1 2 3 4 5 6 7 8 9
 第二次测试:
 删除重复元素后的表: 1 2 3 4 5 6 7 8 9
 第三次测试:
 删除重复元素后的表: 1 2 3 4 5 6 7 8 9

说明：分别测试了无重复、部分连续重复以及高频连续重复的三组数据。程序均能在不遗漏的情况下精准去除多余重复项，且只保留唯一元素，验证了快慢指针原地去重算法的高效与准确。

7 总结、心得和建议

本次实验要求完成顺序表的数个核心功能开发。我独立编写了元素的条件插入、数据的奇偶分离、两表公共元素的提取，以及原表内部的去重操作。软件工程的理论学习需要代码的实验，这次实验提供了一次具体实现的契机。

【心得】

(1)底层物理结构的限制与思考：顺序表在物理层面表现为连续的内存区块，这支持了通过索引快速定位数据，但在插入和删除时会引发大量后续元素的移动。造成浪费，耗时。

(2) 双指针技术：在早期的去重算法中，嵌套循环导致资源消耗呈平方级上升。引入快慢双指针机制后，整个序列仅需被遍历一次，元素整体搬运的动作被彻底消除。同理，在提取公共元素时设置双指针同步推进，单次线性扫描即可完成交集提取，有效降低了算法的时间复杂度。

(3) 模块化开发与防御性编程：依据模块化开发原则，结构体定义和核心算法被独立提取到头文件中，使代码结构变得清晰。同时，在功能函数内部设置针对表满溢出、空表等边界条件的多重防御机制，很大程度上决定了系统的稳定性。

【建议】

(1) 增强对现成模板库的剥离：虽然现成模板库能加速开发，但在学习底层数据结构阶段，应刻意减少依赖，以严谨务实的态度手写每一行底层控制代码，这也是构建复杂应用系统的必然途径。

(2) 引入性能评测指标：常规的正确性输出容易掩盖嵌套循环等算法层面的低效。建议在后续实验中为相关题目设定严格的执行时间上限或操作移动次数限制，从而逼迫开发者主动放弃暴力破解方案，探索时间复杂度最优解。